```
    <input type="button" value="Submit"
id="submit" />
  </body>
  <script src="http://ajax.googleapis.
com/ajax/libs/jquery/2.1.1/jquery.min.
js"></script>
  <script>
    jQuery(document).ready(function($)
{
      var email, pass;
      $('#submit').click(function() {
        email = $('#email').val();
        pass = $('#password').val();

        $.post('/login', { email:
email, pass: pass }, function(data) {
          if (data === 'done') {
            window.location.href = '/
admin';
          }
        });
      });
    });
  </script>
</html>
```

Note: The form inputs (email and password) are not being checked for any sort of validation. The server is simply accepting whatever values the client inputs, as this is a simple demonstration for session creation.

Halt! You have a bug!

Take a moment and look at the above code snippet that we created with the session variable *sess*. What's wrong with it? Did you get it? No?

Well, this variable is a global one. And as we all know, global variables can prove to be terrible security holes in production level projects.

Here, since *sess* is global, the session won't work for multiple users as the server will create the same session for all the users. This can be solved by using what is called a *session store*.

We have to store every session in the store so that each one will belong to only a single user. One popular session store is built using the Redis module.

Session stores can also be created at the server using the already implemented (if any) database management systems like MySQL, MongoDB, PostgreSQl, etc. More information about this can be found on the official express-session npm website *https://www.npmjs.com/package/express-session*.

Let's run the code!

To run the app, simply go to the main directory of the Node project and use the following command:

```
npm index.js
```

This should give you a message that says:

```
App Started on PORT 3000 (or any other
free port)
```

To see exactly what is stored in the cookie, let us use *console.log()* as follows:

```
console.log(req.session);
```

Once the server is running, the home page will look like what's shown in Figure 4.

Here, I have used *testusername* and *password* as test case inputs.

Once the user logs in from the form in Figure 4, the page is redirected to what's shown in Figure 5.

Figure 6a shows what we have on the console.

You can see the number of visits made by the client in Figure 6a. When the page is refreshed, the browser essentially sends a GET request back to the server side with the email variable from the cookie on the client machine. The server reads this and realises that this user is logged in as *testusername*, and thus the variable *session.visits* is only updated for the current logged in session and not globally as each and every request.

Figure 6b shows what the page looks like after a few refreshes.

Figure 7 shows what we have on the console.

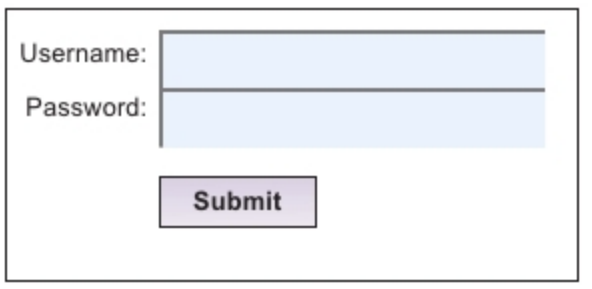

Figure 4: Home page screen with a login form

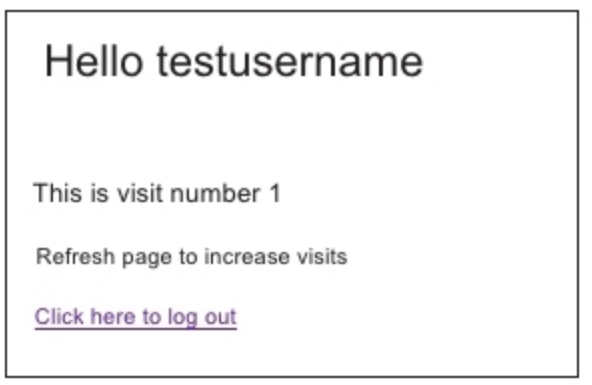

Figure 5: Page after user is logged in

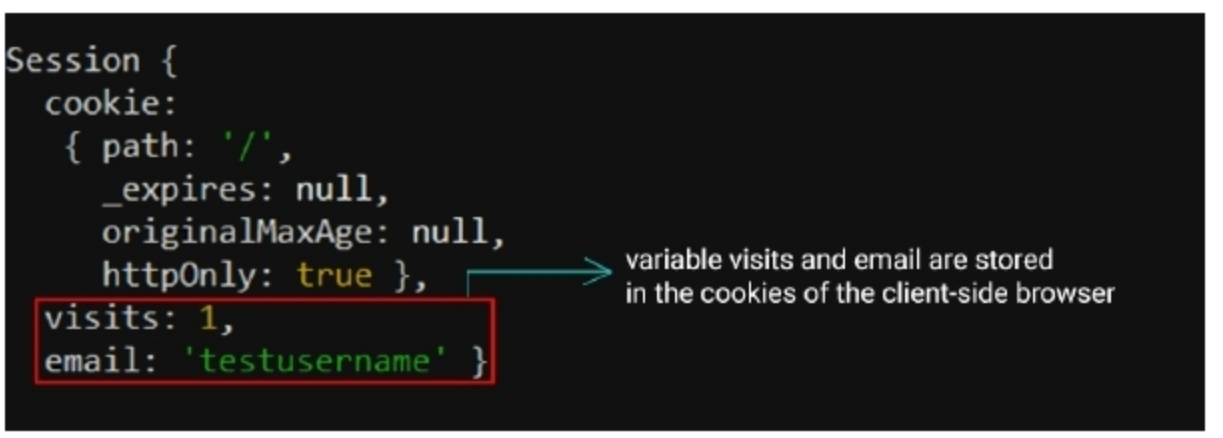

Figure 6a: Console output for *console.log (reg.session)*

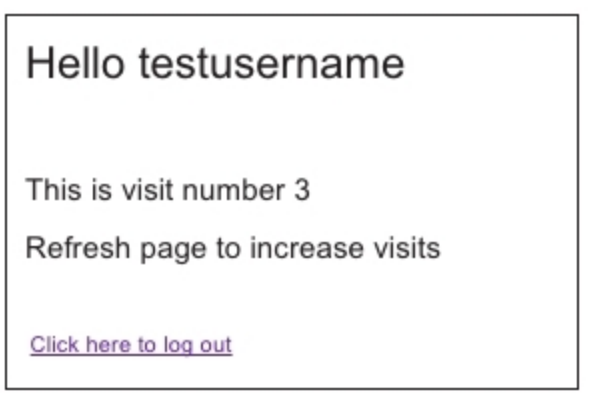

Figure 6b: Page after the user has logged in, along with the visit number

Continued on page 51...